ASSIGNMENT-2(CYBERSECURITY)

Q1-An Operating System is software that acts as an intermediary between users and computer hardware, managing all resources and enabling programs to run efficiently.

The three core resources an OS manages are:

- Process Management: The OS allocates CPU time to different programs. For example, when you open both a web browser and a text editor simultaneously, the OS decides how much processing power each application gets and switches between them seamlessly.

- Memory Management: The OS allocates RAM to running programs and prevents them from interfering with each other. For instance, when you're editing a document in Microsoft Word while streaming a video, the OS ensures each application has its dedicated memory space without crashes.

- File System Management: The OS organizes and controls access to files on your disk drive in a hierarchical structure. For example, when you save a document to your "Documents" folder, the OS manages where that file is physically stored on the hard drive and retrieves it when you open it again.

Q2-The Linux file system follows a hierarchical tree structure starting from the root directory (/). Here are four important directories:

- /etc — Contains system configuration files. This is where settings for services, user accounts, and system-wide configurations are stored. For instance, the SSH server configuration file (sshd_config) is located here.

- /var — Holds variable data that changes frequently, like log files, temporary files, and application data. System administrators check /var/log/ to review system logs and troubleshoot issues.
- /home — The home directory for all regular (non-root) users. Each user has a subdirectory here (e.g., /home/alice/) where they store personal files and configurations.
- /bin — Contains essential command-line programs and utilities that all users need to run (like ls, cat, grep). These are fundamental tools required for basic system operation and are available to every user.

Q3-a) ls -la

This command lists all files in a directory with detailed information including permissions, ownership, size, and date. The -l flag shows long format (detailed view) and -a shows hidden files. Practical use: checking file permissions and ownership before making security changes.

(b) chmod 755 file.sh

This command sets file permissions to read/write/execute for the owner, and read/execute for group and others. It's commonly used to make a shell script executable so users can run it (755 is the standard for executable scripts and programs).

(c) grep 'error' /var/log/syslog

This command searches for the word "error" in the system log file and displays matching lines. Practical use: troubleshooting system problems by quickly finding all error messages in logs without manually reading thousands of lines.

(d) ps aux

This command shows all currently running processes with detailed information including the user running each process, CPU/memory usage, and the command that started it. Practical use: identifying which application is consuming excessive resources or finding a specific process you need to stop.

Q4-The permission string -rwxr-x--- breaks down as follows:

- First character (-): This is the file type; the dash indicates it's a regular file (not a directory).
- Characters 2-4 (rwx) — Owner permissions: The owner can read (r), write (w), and execute (x) this file. They have full control.
- Characters 5-7 (r-x) — Group permissions: Members of the file's group can read and execute the file, but cannot modify it (no write permission).
- Characters 8-10 (—) — Others permissions: All other users have no permissions — they cannot read, write, or execute this file.

Q5-SSH (Secure Shell) is a protocol for securely connecting to remote computers over a network. It encrypts all data transmitted between your computer and the remote server, protecting your credentials and commands from being intercepted.

SSH is preferred over Telnet for two critical security reasons:

1. Encryption: SSH encrypts all communication between client and server, whereas Telnet sends everything in plain text. This means passwords and commands cannot be read if someone intercepts your network traffic. With Telnet, anyone on the network could literally see your login credentials.

2. Public-key Authentication: SSH supports authentication using cryptographic key pairs (public and private keys) instead of just

passwords. This is more secure because your private key never travels over the network, and even if someone captures your key file, it's useless without your passphrase. Telnet only offers weak password authentication.

Q6-To identify and safely terminate a process consuming excessive CPU on a Kali Linux machine:

Step 1: View running processes with live updates

Code-top

This command displays all running processes in real-time, sorted by CPU usage. The process at the top is consuming the most CPU. You'll see the PID (Process ID), user, CPU %, memory usage, and the command name.

Step 2: Get detailed process information (alternative method)

Code-ps aux | grep -v grep | sort -k3 -nr | head -5

This lists the top 5 CPU-consuming processes with full details. The third column shows CPU percentage.

Step 3: Safely terminate the process

Code-kill -15 <PID>

Replace <PID> with the actual process ID. The -15 signal (SIGTERM) gracefully stops the process, allowing it to clean up resources. If it doesn't respond within a few seconds:

Code-kill -9 <PID>

The -9 signal (SIGKILL) forcefully terminates it immediately.

Q7-bash-#!/bin/bash

```
echo "Current date and time: $(date)"

ls /etc > etc_files.txt

echo "Scan complete!"
```

- #!/bin/bash — Shebang line that tells the system to execute this script using the Bash interpreter.
- echo "Current date and time: $(date)" — Prints the text “Current date and time:” followed by the output of the date command, which displays the current date and time in a human-readable format.
- ls /etc > etc_files.txt — Lists all files in the /etc directory and redirects the output (>) to a file named etc_files.txt. This saves the list instead of printing it to the screen.
- echo "Scan complete!" — Prints a completion message to confirm the script finished successfully.

Q8-(a) What does chmod 777 mean?

The command chmod 777 /var/www/html sets all permissions to read, write, and execute for everyone:

- 7 for Owner: read (4) + write (2) + execute (1) = full control
- 7 for Group: read + write + execute = full control
- 7 for Others: read + write + execute = full control

This means literally any user on the system (including attackers with low-level access) can read, modify, and execute any file in this directory.

(b) What are the security risks?

- Unauthorized modification: Anyone can change, delete, or corrupt web application files, potentially injecting malicious code.
- Data exposure: Sensitive files like database credentials or configuration files become readable to all users.
- Malware insertion: An attacker could add backdoors or malware to executable files.
- Loss of accountability: You cannot determine who made changes because everyone has write access.
- Compliance violation: Most security standards (PCI-DSS, HIPAA) require restrictive file permissions; 777 violates these requirements.

(c) Correct chmod command:

chmod 750 /var/www/html

Breaking this down:

- 7 for Owner: Full read, write, execute access
- 5 for Group: read (4) + execute (1) = can view and access files, but not modify
- 0 for Others: No access whatsoever

This ensures only the owner (usually the web server admin) can modify files, the group can read them, and outsiders cannot access them.

Q9-Kali Linux is specifically designed for cybersecurity professionals, whereas standard distributions like Ubuntu or Windows focus on general-purpose computing. Kali comes pre-loaded with hundreds of security testing tools, a hardened kernel optimized for penetration testing, and a philosophy built around offensive security. It’s like the

difference between a toolbox meant for general home repair versus a professional mechanic's specialized toolkit.

Three key pre-installed tools:

1. Nmap (Network Mapper) — A network scanning tool that discovers devices on a network and identifies open ports and running services on target machines. Cybersecurity professionals use it during the reconnaissance phase of penetration tests to understand what's exposed on a network before attempting more detailed attacks.

2. Metasploit Framework — A comprehensive penetration testing platform with thousands of known exploits. It allows testers to simulate real-world attacks against vulnerable systems in a controlled manner. Security teams use it to test whether their systems are vulnerable to known exploits before criminals find them.

3. Wireshark — A network packet analyzer that captures and displays live network traffic in real-time. It shows exactly what data is being transmitted across the network. Cybersecurity professionals use it to detect suspicious traffic patterns, identify malware communication, and understand network behavior during security investigations.

Q10-a) SSH command:

ssh -i id_rsa admin@192.168.1.50

The -i flag specifies the private key file to use for authentication instead of a password.

(b) Public-key authentication explanation:

Public-key authentication uses a pair of mathematically linked keys: a public key (which you share with servers) and a private key (which you keep secret, like a password). When you connect, the server verifies

your private key without ever seeing it. This is more secure than passwords because:

- Your private key never travels over the network
- Even if someone intercepts the connection, they cannot log in without possessing your actual key file
- Private keys can be very long and mathematically complex, making brute-force attacks computationally infeasible
- You can revoke access by removing the public key from the server without changing passwords everywhere

(c) Default port and modified command:

SSH uses port 22 by default. If the server runs SSH on port 2222, use:ssh -i id_rsa -p 2222 admin@192.168.1.50

The -p flag specifies the alternate port number.

Q11-(a) Four investigation commands and what they reveal:

1. ps aux | grep -E "UID|<suspected_process>" or use top

- Reveals: The exact command line used to start the process, parent process ID (PPID), resource consumption, and user who spawned it. This helps determine if the process is legitimate or was launched maliciously.

2. ss -tulnp or netstat -tulnp

- Reveals: All active network connections and listening ports. You can see which process is making outbound connections to the unknown IP address and what port/protocol it's using. This confirms the malicious activity.

3. lsof -p <PID>

- Reveals: All open files, network connections, and resources associated with a specific process. This shows exactly what the suspicious process is accessing (files, network sockets, pipes) and helps understand its purpose.

4. cat /proc/<PID>/status or cat /proc/<PID>/cmdline

- Reveals: Detailed information about the process including memory maps, state, parent relationships, and the exact command used. This helps determine if it's a legitimate system process running under false pretenses.

(b) Conclusions and next steps:

Possible Conclusions:

- The system is likely infected with malware or a rootkit — a malicious program executing automatically without user login, suggesting persistence mechanisms (cron jobs, startup scripts, or kernel-level hooks).

- An unauthorized remote access tool is installed and calling home to a command-and-control (C2) server, allowing an attacker to maintain persistent access.

Next Steps:

1. Immediately isolate the server from the network to prevent the attacker from receiving data or commands.

2. Capture forensic evidence by taking memory dumps (dd if=/dev/mem) and disk images before shutting down.

3. Kill the process to stop ongoing data exfiltration: kill -9 <PID>

4. Investigate persistence by checking cron jobs (crontab -l), startup scripts, and system startup configurations.

5. Perform malware analysis on the binary in a sandboxed environment to understand capabilities and identify the threat actor.

6. Restore from clean backups or completely reinstall the OS after removing the threat.

Q12-The Security Check Script:

```
#!/bin/bash

# Check 1: Display currently logged-in users (detects unauthorized access)
echo "=== Currently Logged-In Users ==="
who

# Check 2: Show all processes running as root (unauthorized privilege escalation attempts)
echo ""
echo "=== Processes Running as Root ==="
ps aux | grep "^root" | grep -v grep

# Check 3: List all files in /tmp older than 7 days (detects old malware artifacts)
echo ""
echo "=== Files in /tmp Older Than 7 Days ==="
find /tmp -type f -mtime +7 -ls 2>/dev/null
```

```
# Check 4: Check if port 22 (SSH) is open and listening (detects unauthorized service modifications)

echo ""

echo "=== SSH Port 22 Status ==="

ss -tlnp | grep :22


echo ""

echo "Security check completed!"
```

Why each check matters from a security perspective:

- User login check: Identifies if unauthorized users have gained access or if accounts are logged in at unusual times.
- Root process check: Detects privilege escalation attempts or malware running with elevated permissions.
- Old /tmp files: /tmp is commonly used by attackers to store tools and malware; old files may indicate past intrusions or persistent backdoors.
- SSH port check: Confirms the SSH service is running and listening on the expected port. If it's not present or on a different port, someone may have compromised it.

Q13-(a) SUID bit explanation and risk:

The SUID (Set User ID) bit is a special permission shown as an s in the owner's execute position (e.g., -rwsr-xr-x). When a file has SUID set, it runs with the privileges of its owner, not the person executing it.

In this case, /usr/local/bin/backup.sh runs as root (the owner) whenever anyone executes it. This is a security concern because:

- A vulnerability in the script could let an attacker execute arbitrary code as root, giving them complete system control.
- If the script is poorly written or calls other programs unsafely, attackers can exploit this to bypass normal permission restrictions.
- The risk is amplified because backup scripts often have legitimate reasons to access sensitive files, but a flaw could be weaponized.

(b) Abuse technique in plain English:

The attacker would exploit a relative path vulnerability. Here's the attack:

If the backup.sh script contains a line like:

tar czf backup.tar.gz /important/data

But it should use the full path:

/bin/tar czf backup.tar.gz /important/data

The attacker can create a malicious tar program in a directory they control (like their home directory or /tmp), then manipulate the system's PATH variable (the list of directories where the system searches for commands) to make the system execute the attacker's fake tar instead of the real one.

Since the script runs as root, the attacker's malicious tar program now executes with root privileges. The attacker's version can do anything — create root backdoors, steal data, or install malware — while appearing to be a legitimate system process.

(c) How to fix it:

1. Remove the SUID bit:

chmod u-s /usr/local/bin/backup.sh

1. Use absolute paths for all commands in the script (never relative paths):

/bin/tar czf /backup/data.tar.gz /important/data

/usr/bin/find /tmp -type f -delete

1. Restrict execution: Only allow specific users or groups to run the script:

chmod 750 /usr/local/bin/backup.sh

chown root:admin /usr/local/bin/backup.sh

4. Review the script to eliminate any dynamic command execution or user input that could be manipulated.

Q14-*/5 * * * * /tmp/.hidden/backdoor.sh > /dev/null 2>&1

- */5 (minute) — Every 5 minutes
    - * (hour) — Every hour (all hours)
    - * (day of month) — Every day
    - * (month) — Every month
    - * (day of week) — Every day of the week

This runs every 5 minutes, 24/7, without any exception. So the backdoor executes 288 times per day (60 minutes ÷ 5 × 24 hours).

(b) /dev/null 2>&1 explained:

- > /dev/null — Redirects standard output (stdout) to /dev/null, which discards it. The program output goes nowhere.
- 2>&1 — Redirects standard error (stderr, file descriptor 2) to wherever standard output (file descriptor 1) is going, which is also /dev/null.

Why an attacker adds this: To hide the script's activity from system logs. If errors or output were logged, a system administrator checking logs might notice the suspicious activity. By silencing all output, the attacker makes the backdoor invisible in normal log reviews.

(c) Two security red flags:

1. Hidden file in /tmp — Files starting with a dot (.hidden) are hidden from normal ls output, suggesting deliberate concealment. Additionally, /tmp is meant for temporary files; a persistent backdoor should never be stored there. An attacker hides it here to evade detection during routine directory scans.

2. Frequent execution every 5 minutes — A legitimate system task might run daily, weekly, or hourly, but every 5 minutes is suspicious. This ensures the backdoor restarts quickly if killed, or maintains persistent access if the attacker needs to reconnect. No legitimate system process needs to run this frequently with output silenced.

Q15-Command:

sudo usermod -s /usr/sbin/nologin webapp

sudo usermod -L root

Security risk addressed: By default, service accounts might have shell access, allowing attackers who compromise the web application to execute arbitrary commands. The nologin shell restricts them to the application only. Locking the root account prevents direct login attempts.

What happens if skipped: A compromised web application user account would grant an attacker full shell access, escalating from application-level compromise to system-wide compromise. An

attacker could brute-force the root account or use captured credentials.

2. Disable Unnecessary Services

Command:

sudo systemctl disable avahi-daemon

sudo systemctl disable cups

sudo systemctl stop avahi-daemon

sudo systemctl stop cups

Security risk addressed: Every running service is a potential attack surface. Avahi (mDNS) and CUPS (print service) are unnecessary for a web server and create unnecessary vulnerabilities.

What happens if skipped: Attackers could exploit unneeded services to gain initial access or lateral movement within your network. Each disabled service reduces your attack surface.

3. SSH Configuration Hardening

Command/Configuration:

sudo nano /etc/ssh/sshd_config

Make these changes:

PermitRootLogin no

PasswordAuthentication no

PubkeyAuthentication yes

Port 2222

X11Forwarding no

MaxAuthTries 3

Then restart:

sudo systemctl restart ssh

Security risk addressed: Weak SSH configuration is the #1 entry point for attackers. Disabling password auth, root login, and changing the default port eliminates the most common attack methods.

What happens if skipped: Your server becomes vulnerable to brute-force password attacks, root account takeover, and automated scanners that target port 22. Attackers worldwide constantly scan for open SSH ports with weak credentials.

4. File Permissions and Ownership

Command:

sudo chown -R root:webapp /var/www/html

sudo chmod 750 /var/www/html

sudo chmod 640 /var/www/html/*.php

sudo chmod 640 /var/www/html/config.php

Security risk addressed: Misconfigured file permissions allow unauthorized users or the web server process to modify code (code injection) or read sensitive configuration files containing database credentials.

What happens if skipped: An attacker who compromises the web application could modify PHP files to inject malware, escalate privileges, or modify application logic. Configuration files could be read to extract database credentials.

5. Install and Configure Firewall

Command:

sudo ufw enable

sudo ufw default deny incoming

sudo ufw default allow outgoing

sudo ufw allow 22/tcp

sudo ufw allow 80/tcp

sudo ufw allow 443/tcp

sudo ufw allow 2222/tcp

sudo ufw status

Security risk addressed: Without a firewall, all ports are exposed to the internet, multiplying attack surface. Attackers can probe for vulnerabilities on any exposed service.

What happens if skipped: Unnecessary ports and services become accessible to attackers worldwide. Even if a service isn't running, open ports indicate the OS version and services present to reconnaissance tools like Nmap.

6. System Updates and Patch Management

Command:

sudo apt update

sudo apt upgrade -y

sudo apt install unattended-upgrades

sudo nano /etc/apt/apt.conf.d/50unattended-upgrades

Uncomment: Unattended-Upgrade::Automatic-Reboot "true";

Security risk addressed: Unpatched systems contain known vulnerabilities that attackers actively exploit. Many breaches succeed because organizations don't patch promptly.

What happens if skipped: The server is vulnerable to public exploits. Once a vulnerability is published, attackers develop working exploits within days. Your unpatched server is a sitting duck for automated attack tools that scan and exploit known CVEs.

7. Audit Logging Configuration (Bonus)

Command:

sudo apt install auditd

sudo systemctl enable auditd

sudo auditctl -w /var/www/html/ -p wa -k webapp_changes

Security risk addressed: Without audit logs, you won't detect compromises or investigate incidents. Logs are crucial for forensics.

What happens if skipped: If your server is compromised, you have no record of what the attacker did, when they accessed it, or what was changed. You cannot determine the scope of the breach.